

Name : Jana Mostafa Mohamed Abdelaal

Major : Systems and Biomedical Engineering

Generative AI Technical Questions

1.

A generative model learns how the data itself is put together — it models the underlying distribution of the inputs (and often the joint distribution of inputs and labels), which means it can produce new, realistic samples. A discriminative model doesn't bother learning that full picture; it only learns the boundary or mapping needed to tell classes apart, i.e. it models the conditional probability of a label given an input. A GAN or GPT-style language model is generative, since both can generate new images or text. A logistic regression or a standard image classifier (like a CNN trained to output "cat" or "dog") is discriminative — it only ever draws boundaries between classes, it never learns to produce a new cat photo.

2.

Transformers exist to model relationships between elements of a sequence without processing them one at a time the way RNNs do. That sequential bottleneck was the main limit on both training speed and how well older models could capture long-range dependencies. A transformer instead uses self-attention: every token is compared against every other token in the sequence at once, producing a set of attention weights that say how much each token should "pay attention to" the others when building its representation. Stack several of these attention layers (plus feed-forward layers and positional encodings, since attention itself has no sense of word order) and you get a model that can be trained in parallel and captures dependencies regardless of how far apart two related words are in the text.

3.

Supervised learning trains on labeled examples — the model is shown inputs paired with the correct output and learns to reproduce that mapping, e.g. predicting whether an image contains a cat. Unsupervised learning works with unlabeled data and tries to find structure on its own, like clustering similar items together or reducing dimensionality to find patterns nobody explicitly pointed out. Reinforcement learning is different again: there's no fixed dataset of right answers, just an agent interacting with an environment, taking actions, and getting a reward signal. It learns a policy by trial and error, trying to maximize cumulative reward over time rather than matching a labeled target.

4.

Tokens are the actual units of text an LLM processes — not always full words, often subword pieces, produced by a tokenizer (commonly something like byte-pair encoding). "Unbelievable" might become "un", "believ", "able", for instance. They matter for a few practical reasons: the model's context window is measured in tokens, not characters or words, so token count directly affects how much text fits in a prompt and how much an API call costs. Subword tokenization also lets the model handle rare or unseen words gracefully by breaking them into familiar pieces instead of treating them as a single unknown symbol.

5.

Attention lets a model decide, for each token, how much weight to give every other token when forming that token's representation. Concretely, each token is projected into a query, a key, and a value vector; the query is compared against every key to produce similarity scores, those scores are turned into weights via softmax, and the output is a weighted sum of the value vectors. Practically, this means a word at the start of a paragraph can directly influence how a word at the end is interpreted, without having to pass information step-by-step through every token in between the way an RNN would. That directly addresses long-range dependency problems and vanishing gradients, and because attention doesn't require sequential processing, it also enables much better parallelization during training.

6.

Prompt engineering is the practice of designing the wording, structure, and examples in a prompt so a model produces the output you actually want, without changing any of its weights. It's crucial because LLM output quality is genuinely sensitive to how a request is phrased — the same underlying question asked two different ways can produce noticeably different answers — and for most people building on top of an existing model, the prompt is the main lever available to steer behavior. Common types include zero-shot prompts (just the instruction, no examples), few-shot prompts (a handful of examples showing the desired input/output pattern), chain-of-thought prompts (asking the model to reason step by step), system/role prompts (setting the model's persona or constraints up front), and instruction prompts (direct, explicit commands about the task).

7.

RAG combines a retrieval step with a generation step so the model can answer using information it wasn't necessarily trained on. When a query comes in, it's converted into an embedding and compared against a store of pre-embedded document chunks to find the most semantically relevant ones. Those retrieved chunks get inserted into the model's prompt alongside the original question, and the model generates its answer grounded in that retrieved context rather than relying purely on what it memorized during training. This is what lets an LLM answer questions about, say, a company's internal documents or yesterday's news, without retraining the model at all.

8. An embedding is a numeric vector representation of a piece of data — text, an image, audio — placed in a continuous space where semantic similarity roughly corresponds to geometric closeness. For retrieval, this matters because it lets you compare meaning rather than exact wording: a query and a document chunk can be matched by cosine similarity or dot product even if they don't share a single keyword, as long as they mean similar things. That's the core mechanism that makes semantic search, and by extension RAG, possible.

9.

Temperature scales the probability distribution the model samples the next token from. A low temperature sharpens the distribution toward the highest-probability tokens, making output more deterministic, focused, and repeatable — useful when you want a factual or consistent answer. A high temperature flattens the distribution, giving lower-probability tokens a real chance of being picked, which produces more varied and sometimes more creative output, but at the cost of coherence and predictability. At temperature 0 the model essentially always picks the single most likely token, i.e. greedy decoding.

10.

Chain-of-thought prompting asks the model to write out its intermediate reasoning steps before giving a final answer, instead of jumping straight to a conclusion. It improves reasoning on multi-step problems because it effectively gives the model more "space" to work through sub-problems sequentially, similar to how a person solving a hard problem on paper works better than trying to do it entirely in their head. It also makes errors easier to catch, since a wrong intermediate step is visible rather than hidden inside a single jump to an answer.

11.

RAG adds external knowledge to a model at inference time by retrieving and injecting relevant context into the prompt, without touching the model's weights at all. Fine-tuning actually adjusts the model's parameters by training further on a task-specific dataset, changing its behavior, style, or domain knowledge permanently. RAG is the better fit when the knowledge base is large, changes often, or needs to stay easily updatable — a document Q&A system over a company's internal wiki, for example. Fine-tuning is the better fit when you need consistent behavior baked in that doesn't depend on retrieval quality — for instance, training a model to reliably output a fixed JSON schema, or to consistently write in a specific tone regardless of what's in the prompt.

12.

A naive RAG pipeline — embed the query, do a single similarity search, stuff the top results into the prompt — has plenty of room for improvement on both sides of the retrieval step.

Pre-retrieval:

- Query rewriting or expansion, so a poorly phrased or underspecified user question gets turned into a better search query before retrieval runs
- Smarter chunking — splitting by semantic boundaries (headings, sections) instead of a fixed character count, so chunks don't cut a table or a procedure in half

Post-retrieval:

- Re-ranking the retrieved chunks with a more accurate (but slower) cross-encoder model, since the initial similarity search is optimized for speed over precision
- Filtering out low-relevance chunks below a score threshold instead of always taking a fixed top-k

13.

Accuracy on the specific task matters most, since a cheaper or faster model is worthless if it gets things wrong. Beyond that: latency, since a slow response breaks a conversational experience; cost per token, especially at scale; context window size, which limits how much retrieved context or conversation history you can include; privacy and data residency requirements, particularly if the application handles sensitive or proprietary data; and reasoning ability, which matters more for multi-step or technical tasks than for simple lookups. Practical factors like function/tool-calling support, rate limits, and how stable the provider's availability is also end up mattering once you're actually running something in production.

14.

The core check is whether every claim in the answer can actually be traced back to something in the retrieved sources, rather than being invented. This can be done a few ways: attribution or citation checking, where the answer is required to reference specific source spans and those references are verified programmatically; an LLM-as-judge setup, where a separate model call compares the answer against the retrieved context and flags unsupported claims; or entailment-style scoring, checking whether each sentence in the answer is logically supported by the retrieved text. In practice a combination — automated groundedness scoring plus periodic manual spot-checks — tends to catch more than either approach alone.

15.

There's a few natural points of overlap. Vision-language models let a system take an image directly as input alongside text — for example, a user could photograph a part and ask what it is or whether it looks damaged, and the model reasons over both the image and the question together. Vision can also feed a generative pipeline indirectly: OCR or object detection can pull structured information out of a photo (a label, a serial number, a gauge reading) and hand that as clean text to an LLM-based assistant, rather than asking the LLM to interpret raw pixels itself. And generative vision models (diffusion-based image generation) can go the other direction, producing visual content from a text description — useful for things like generating reference diagrams or visualizing a described design change.

16. Design an AI assistant for CURT.

a. System architecture

Rather than drawing this as an image, here's the pipeline component by component, in the order a request actually flows through it:

- Ingestion layer — connectors that pull in documents from wherever each team already keeps them (shared drives, local uploads), with parsers for the different file types each team produces: PDFs and

Word docs for reports, spreadsheets for the business team, CAD exports and schematics for mechanical/electrical, code and markdown for software.

- Preprocessing and chunking service — normalizes everything into clean text (including OCR or a vision model for scanned diagrams/schematics), splits it into chunks along semantic boundaries, and tags each chunk with metadata: which team it belongs to, file type, date, author.
- Embedding + vector store — embeds each chunk and stores it alongside its metadata, so retrieval can be both semantic and filtered.
- Structured data layer — a separate access path for structured data like inventory counts or telemetry logs, exposed through defined query functions rather than letting anything query the database directly.
- Access control layer — authenticates the user and tags their request with their team/role, so retrieval only ever searches documents that team is allowed to see.
- Retrieval and orchestration layer — the core RAG engine. Given a query, it decides whether the answer needs document retrieval, a structured-data tool call, or both, and assembles the context to send to the model.
- The LLM — generates the answer from the retrieved context and/or tool results, instructed to answer only from what it was given and to cite where each part of the answer came from.
- Citation/verification layer — attaches source references to the final answer so a user can check it against the original document.
- Frontend — a chat interface, most likely per-team or with a team filter, so members interact with their own scope by default.
- Logging and evaluation layer — records queries, what was retrieved, and what was answered, both for auditing and for building an evaluation set over time.

b. Main challenges

- Data preprocessing and file formats: four different teams produce very different document types — CAD files and schematics, spreadsheets, code, written reports — and diagrams in particular need OCR or a vision model rather than a plain text extractor, since useful information often lives in an image, not the surrounding text.
- Inconsistent, incomplete, or noisy data: outdated or duplicate versions of the same document, missing metadata, and inconsistent formatting all directly hurt retrieval quality, since the system has no way to know which version is current unless that's tracked explicitly.
- Chunking and context limits: a long design report has to be split into pieces that fit the model's context window, but splitting naively can cut a table or a multi-step procedure in half, losing the information that made that chunk useful in the first place.
- Retrieval relevance: CURT-specific jargon and part names may not embed well with a general-purpose embedding model, and a query can legitimately span more than one team's documents, making it harder to know what to retrieve.
- Accuracy of generated answers: the usual RAG risk — the model can still hallucinate, especially if the retrieved context is incomplete, contradictory across document versions, or simply doesn't contain the answer.

c. Techniques to address these

- Use format-aware parsers per file type, plus OCR or a vision-capable model for schematics and diagrams, so everything gets normalized into consistent text before it's chunked — this directly addresses the file-format and noisy-data challenges.
- Use hybrid retrieval — dense embedding similarity combined with keyword/BM25 search — plus a re-ranking step, so exact terms like part numbers aren't lost the way they can be with embeddings alone, and the most relevant chunks actually end up at the top before being sent to the model. This addresses the retrieval-relevance and accuracy challenges together.
- Semantic chunking that respects document structure (keeping tables and step-by-step procedures intact) rather than fixed-length splitting, to reduce the chunking problem specifically.

- Require the model to answer strictly from retrieved context, explicitly say when it can't find an answer, and cite sources for every claim — this won't eliminate hallucination but makes it far easier to catch when it happens.

d. Evaluating accuracy and reliability

I'd build a small labeled test set of realistic questions per team, each with a known correct answer and the document it should come from. Against that set, I'd track retrieval metrics (did the system actually pull back the right source chunk, measured as recall/precision at k), groundedness (does the generated answer's content actually match what's in the cited sources — checkable with an LLM-as-judge or manual spot review), and end-to-end answer correctness against the known-good answers. In production, I'd also keep lightweight user feedback (a thumbs up/down on each answer) and periodically review the flagged ones. Any time the chunking, retrieval, or model setup changes, I'd re-run the test set as a regression check before rolling the change out, rather than assuming a change that looks reasonable didn't quietly hurt accuracy somewhere else.